

Navigators Technical Documentation

Navigators — Technology Stack Reference

Document Identifier: STACK-NAV-01
Status: Production Reference
Version: 1.0.0
Last Updated: 2026-09-23
Attribution: Developed by Navigators

1. Architecture-Wide Technology Matrix

Layer	Technology	Primary Role in Repository	Technical Justification
Edge Navigation Runtime	Vanilla JavaScript (ES2022)	Core edge engine (simulator/app.js , simulator/engine/)	Maximum execution speed, zero transpile overhead, direct browser WebAssembly interop.
Edge UI Framework	Vanilla HTML5 / Modern CSS	Navigation HUD and telemetry dashboard	Zero framework runtime footprint, instant DOM updates, high battery efficiency on mobile devices.
Edge Neural Engine	ONNX Runtime WebAssembly	Neural velocity inference in browser (ort.wasm.min.js)	Executes exported PyTorch TCN models client-side with SIMD vectorization and zero server calls.
Edge Map Rendering	Leaflet.js (v1.9.4)	Offline vector and tile map display	Lightweight (42KB), robust tile layer caching, low memory consumption on mobile browsers.
Edge Offline Cache	HTML5 Service Worker API	Static asset and WASM binary caching (simulator/sw.js)	Guarantees full offline application boot without cellular network access.
Edge Local Storage	Web Storage API (LocalStorage/IndexedDB)	Road network geometries and offline sync queue	Instant persistence of vector road geometries and asynchronous contribution queues.
Backend API Server	FastAPI (Python 3.14)	RESTful API service (src/api/server.py)	High-performance asynchronous routing, native Pydantic validation, clean OpenAPI generation.
ASGI Server	Uvicorn	Production ASGI web server	Asynchronous request processing, SSL/TLS certificate termination for mobile pairing.
Relational Database	SQLite 3	Relational state store (src/db/schema.sql , data/navigators.db)	Zero-configuration embedded database, ACID transactional integrity via WAL, zero server daemon overhead.
Deep Learning Engine	PyTorch (v2.x)	Model training & evaluation (src/models/ , scripts/train.py)	Industry standard tensor computing, native support for causal 1D dilated convolutions, MPS/CUDA acceleration.

Layer	Technology	Primary Role in Repository	Technical Justification
Model Serialization	ONNX (Open Neural Network Exchange)	Edge artifact export (<code>src/edge/onnx_export.py</code>)	Interoperable cross-platform graph representation executable via WebAssembly on mobile browsers.
Scientific Computing	NumPy & SciPy	Matrix math, EKF, Butterworth signal filtering	High-performance vectorized numerical routines for baseline validation and replay evaluation.
Data Analysis	Pandas	IO-VNBD dataset parsing and trajectory analysis	Efficient tabular data manipulation and session-level train/validation splitting.
Automated Testing	Pytest & AnyIO	Python backend, RBAC, and algorithm testing	Fixture-based test runner, parameterized security matrix validation, isolated temp databases.
Edge Engine Testing	Node.js Built-in Test Runner (<code>node --test</code>)	JavaScript EKF and motion logic validation	Zero-dependency testing of client-side mathematical routines in pure Node.js environments.
Security Cryptography	Python <code>hashlib</code> & <code>secrets</code>	Token generation, SHA-256 hashing, password salts	Cryptographically secure pseudo-random tokens and secure one-way credential hashing.

2. Layer-by-Layer Justifications

2.1 Edge Runtime Tier (Smartphone Browser)

Vanilla JavaScript (ES2022) & WebAssembly

- **Why It Is Used:** The client-side navigation engine must operate inside low-to-mid-range smartphone browsers without thermal throttling or memory spikes. Vanilla JavaScript avoids the memory and garbage-collection overhead of heavy single-page application frameworks like React or Angular.
- **Implementation:** Located in `simulator/engine/` (`ekf.js` , `alignment.js` , `map_matcher.js` , `pedestrian.js` , `preprocessing.js`).

ONNX Runtime Web (`ort.wasm.min.js`)

- **Why It Is Used:** Rather than streaming sensor telemetry to a remote server for AI inference (which introduces network latency and fails completely in GNSS-denied tunnels), ONNX Runtime Web executes the neural network locally on the phone’s CPU via WebAssembly SIMD.

- **Implementation:** Vendor binaries stored in `simulator/vendor/onnxruntime/`.

Service Worker API (`simulator/sw.js`)

- **Why It Is Used:** Enables True Offline operation. Once visited, all HTML, CSS, JavaScript, WebAssembly binaries, model weights (`model.onnx`), and road geometries are cached locally, allowing the application to launch and navigate even when the phone is in Airplane Mode.

2.2 Backend & Data Processing Tier

Python 3.14 & FastAPI

- **Why It Is Used:** Python provides seamless integration with the scientific computing ecosystem (PyTorch, NumPy, SciPy). FastAPI provides high-speed asynchronous request routing with automatic request validation through Pydantic models.
- **Implementation:** Located in `src/api/` with modular routers for authentication, moderation, datasets, models, and places.

SQLite with Write-Ahead Logging (WAL)

- **Why It Is Used:** The backend architecture is designed for streamlined deployment without requiring heavy external database services like PostgreSQL or MySQL. SQLite operates embedded within the application process, offering microsecond read latency and transactional safety with `PRAGMA journal_mode=WAL;`.
- **Implementation:** Schema defined in `src/db/schema.sql`; database helper in `src/db/database.py`.

PyTorch & Temporal Convolutional Networks

- **Why It Is Used:** Vehicle speed estimation from IMU data is inherently a sequential temporal problem. PyTorch enables custom causal 1D convolutions with strict left padding, preventing future-sample leakage during training.
- **Implementation:** Model definition in `src/models/tcn_model.py`; training engine in `src/models/trainer.py`.

Developed by Navigators